Chapter 10 exercises

DS10-E01 • Optimal BST interval DP

Successful probabilities are `[0.1,0.2,0.7]`, with no failed queries. Fill C[i, j] and optimal roots for lengths 1,2,3 and reconstruct the tree. Explain why adding q probabilities requires empty costs consistent with the comparison convention.

DS10-E02 • AVL insertion versus deletion

Insert 30,10,20 into an empty AVL tree. List balance factors, rotations, and heights. Construct a deletion whose unbalanced node has a taller child with bf=0. Does it require a single or double rotation? Why is the original insertion direction insufficient?

DS10-E03 • Red-black black height and insertion

In a general node-colored red-black tree, insert 10,5,1,7. List recolorings and rotations. Check equal black counts on all root-to-NIL paths. Must general red-black validity imply LLRB validity?

DS10-E04 • Splay amortized bounds

Start with the right-chain BST 1→2→3→4. Search for 4 and draw each bottom-up zig-zig or zig. Explain why a Θ(n) search does not contradict amortized O(log n), and show the initial-potential term for an arbitrary starting tree.

DS10-E05 • Local proof of red-black deletion repair

The deficit x is left of parent p. Black sibling w has a red far nephew and any valid near-nephew color. Explain why giving w p's original color, coloring p and the far nephew black, and left-rotating p removes the deficit. Cover both original colors of p.

DS10-E06 • Successor transplant, color, and element identity

Use a general red-black tree with black root 20, black left child 10, and black right child 30 whose children are red 25 and red 40. All remaining children are black NIL. Delete 20 using complete node transplant. Identify y, x, removed_color, and every reconnected link. Decide whether delete_fix runs and verify final black counts. How would copying successor contents affect a promise that handles to undeleted elements remain stable?

Lin Cing https://github.com/xinhi6716/computer-science-self-study

DS10-E07 • Verify double rotations with sizes and potential

Every node has weight 1; s includes the node, each rotation costs 1, and $\Phi=\Sigma\log_2 s$. T(k,L,R) denotes a BST with root key k; a standalone number denotes a leaf.

(1) Splay 10 in T(30,T(20,T(10,5,15),25),35). Give rotations, s(10), s(20), s(30) in all three states, and endpoint potentials.
(2) Splay 20 in T(30,T(10,5,T(20,15,25)),35). Repeat the accounting; compare amortized cost with $2\log_2(7/3)$.
(3) Identify the disjoint node sets used by concavity. If case (1) instead right-rotates 20, then 30, is the final tree identical? Can the same zig-zig disjoint-set argument still be applied?

DS10-E08 • AVL height and node count

Take empty-tree height −1 and leaf height 0. Prove the minimum-node recurrence for height h. Find the maximum height of a 200-node AVL tree and show that it is attainable. Convert the answer to the leaf-height-one convention.

DS10-E09 • Red-black and multiway trees

Draw both general red-black encodings of a 3-node with a<b, and the encoding of a 4-node with a<b<c. Explain why contraction produces nodes with at most three keys and external leaves of equal depth.

DS10-E10 • Comparison costs and rotation counts

A single key k_1 has probability $p_1=.4$; the two unsuccessful-query probabilities are $q_0=.1$ and $q_1=.5$. Find expected cost counting only real-key comparisons and then counting one additional dummy visit. For AVL insertion of 30, 10, 20, relate a double rotation to primitive rotations.

References and attribution

Lin Cing, handwritten data structures notes. Sleator and Tarjan, Self-Adjusting Binary Search Trees, JACM 32(3), 1985, § 2: bottom-up splay and the access lemma. Numeric diagrams use this chapter's own keys and unit weights. The MIT 6.006 2020 index, L7, identifies AVL material. Princeton balanced search trees uses LLRB and its insertion/deletion code must not be mixed with general red-black conventions.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.
https://github.com/xinhi6716/computer-science-self-study